

UNIVERSIDAD NACIONAL DE PILAR
TECNICATURA UNIVERSITARIA
EN DESARROLLO DE SOFTWARE

WEB DESARROLLO FULL STACK

[2 0 2 6]

Entendiendo el DOM

Cuando abrimos una página web, el navegador no lee el archivo **HTML** como si fuera un simple bloque de texto de arriba a abajo. Para poder dibujar los paneles, los botones y los cuadros de texto de nuestro tablero, el navegador ejecuta un proceso de ingeniería llamado **parseo** (*parsing*) -**Ver al final en extenso- y construye el **DOM**.

Tenemos que comprender cómo opera esta estructura interna, para poder capturar los datos de la interfase e inyectar las alertas que nos envíe nuestro búnker central. Vamos a desarmar este concepto clave paso a paso.

DOM

Las siglas significan **Document Object Model** (Modelo de Objeto de Documento).

Tengamos en cuenta lo fantástico y evolucionado de este concepto, tan diferente a un compilador o a un intérprete:

La definición técnica más simple es esta: **El DOM es una copia estructural y en RAM que el navegador hace de nuestro archivo HTML, pero transformada en un MAPA DE OBJETOS MANIPULABLES.**

Cuando el navegador carga nuestro código, toma cada etiqueta (**<div>**, **<button>**, **<input>**, ****) y la convierte en un "nodo" o pieza de software independiente.

El Árbol Estructural

El **DOM** organiza estas piezas en una estructura jerárquica de árbol (padres, hijos, nietos, etc). Si un botón está adentro de un panel, en el **DOM** ese elemento botón va a ser un elemento hijo que depende directamente del nodo padre panel.

TEXTO ESTÁTICO VS. OBJETOS DINÁMICOS

Tenemos que entender la diferencia crítica entre el archivo fuente y el entorno de ejecución:

El archivo HTML es un texto estático. Es un documento quieto grabado en el disco. JavaScript no puede interactuar con una línea de texto de un archivo guardado para cambiarle el color en vivo.

El DOM es un objeto vivo en memoria RAM. Es la representación gráfica activa. Si modificamos el **DOM** mediante programación, la pantalla de nuestra computadora se altera de forma inmediata en el mismo milisegundo, sin necesidad de recargar la página.

Cuando en nuestro script escribimos el objeto global **document**, le estamos ordenando al navegador: *"Ingresa al mapa vivo de la pantalla actual para alterar sus propiedades a tiempo real"*.

Las Dos Operaciones Clave en Nuestro Proyecto

En nuestro desarrollo del *Target Analyzer* realizamos únicamente dos acciones sobre este mapa interactivo: **Escuchar Sensores** e **Injectar Resultados**.

CAPTURA Y ESCUCHA (LEER EL DOM)

Para reaccionar cuando hacemos clic en el botón de escaneo, primero tenemos que localizar esa pieza exacta dentro del mapa de memoria del navegador. Para eso usamos comandos de búsqueda selectiva:

document.getElementById('btn-scan')

Le ordenamos al navegador que recorra todo el árbol del **DOM**, busque el nodo único que tenga el atributo **id="btn-scan"** y nos devuelva el control de esa pieza.

document.querySelector('#panel-tech .contenido-panel')

Es un buscador más avanzado. Rastrea el mapa buscando un contenedor que tenga el ID panel-tech (un sub-árbol con padre panel-tech) y, adentro de él, una pieza con la clase .contenido-panel.

Una vez que capturamos la pieza y la guardamos en una constante (como **botonEscaneo**), le aplicamos un *escuchador* (**addEventListener**). Esto deja al nodo en un estado de alerta física, esperando que interactuemos con él.

MODIFICACIÓN E INYECCIÓN (ESCRIBIR EN EL DOM)

Cuando nuestro **servidor backend** responde con éxito, necesitamos actualizar nuestro tablero de control de forma dinámica. **Aquí es donde alteramos las propiedades internas del objeto vivo mediante la instrucción .innerHTML.**

Por ejemplo, al escribir **panelTech.innerHTML = '[ENVIANDO PAQUETE]';**

No estamos modificando nuestro archivo **.html** físico del disco duro. Le estamos ordenando al motor del navegador: *"Borrá de forma inmediata lo que tenga adentro el nodo panelTech en la memoria RAM e inyectale esta nueva etiqueta en tiempo real"*.

El navegador obedece, redibuja el sector específico de la interfase y nosotros vemos el cambio de estado en la pantalla al instante.

Resumen de Flujo de información y conocimiento.



Nuestra cadena de causalidad operativa con el DOM se resume de punta a punta en este ciclo cerrado:

Comprendiendo este puente, dejamos en claro que **JavaScript no habla con la pantalla directamente, sino que manipula el DOM, y el navegador es el encargado de reflejar esa manipulación en nuestros monitores.**

Necesitamos ver exactamente qué líneas actúan para leer el mapa de la pantalla y qué líneas actúan como "inyectores" para modificar la pantalla en tiempo real.

El DOM en Acción (script.js)

Ya sabemos que el DOM es la copia interactiva en memoria RAM que el navegador hace de nuestra pantalla. Ahora vamos a identificar de forma exacta cuáles son las instrucciones de nuestro archivo script.js que tocan, leen o alteran esta estructura.

LA FASE DE MAPEO

Al arrancar el script, lo primero que hacemos es crear constantes para guardar las referencias de las piezas del DOM que vamos a necesitar manipular. Esto nos evita tener que recorrer todo el árbol del navegador cada vez que queramos hacer un cambio.

Línea 2

```
const inputObjetivo = document.getElementById('target-url');
```

Operación de lectura/mapeo.

Invocamos al objeto global **document** (*la puerta de entrada al **DOM***) y usamos el método **getElementById**. *Le ordenamos al navegador que busque la etiqueta exacta que tenga **id="target-url"***. A partir de este renglón, la *constante* **inputObjetivo** pasa a ser el control remoto de ese cuadro de texto en la pantalla.

Línea 3

```
const botonEscaneo = document.getElementById('btn-scan');
```

Operación de lectura/mapeo.

La misma lógica analítica. Buscamos la pieza única con el identificador del botón de escaneo y guardamos su control en la *constante* **botonEscaneo**.

Líneas 4, 5 y 6

```
const panelVista = document.querySelector('#panel-vista .contenido-panel ');  
const panelTech = document.querySelector('#panel-tech .contenido-panel ');  
const panelEnlaces = document.querySelector('#panel-enlaces .contenido-panel ');
```

Operación de lectura/Mapeo avanzado.

Aquí usamos **document.querySelector**, un sensor más potente que busca por selectores CSS idénticos a los que usamos en nuestras hojas de estilos. Le indicamos al navegador: *"Buscá el contenedor con el ID panel-vista (o panel-tech, o panel-enlaces) y, en su interior, localizá el nodo hijo que tenga la clase .contenido-panel"*. Almacenamos estas tres secciones específicas de nuestro tablero de control en sus respectivas constantes.

LA ESCUCHA (LOS SENSORES).

Línea 8

```
botonEscaneo.addEventListener('click', iniciarOperacion);
```

Operación de vinculación de evento en el DOM.

Tomamos el nodo del botón que mapeamos en la línea 3 y le acoplamos un "escuchador de eventos" (**addEventListener**). *Esto altera el estado físico de esa pieza en el **DOM**: el navegador la pone en un estado de alerta continuo, monitoreando la placa del mouse.*

*En el momento exacto en que hacemos clic sobre ese sector de la pantalla, el **DOM** reacciona y gatilla de forma automática nuestra función iniciarOperacion.*

LA EXTRACCIÓN DE DATOS

Línea 11

```
const urlIngresada = inputObjetivo.value;
```

Operación de lectura de propiedad del **DOM**.

No estamos alterando la pantalla; estamos extrayendo información de ella. Accedemos a la constante inputObjetivo (que gobierna nuestro cuadro de texto) y leemos su propiedad nativa .value. El **DOM** nos entrega la cadena de caracteres cruda que escribimos en ese instante en la interfase, y la guardamos en la variable urlIngresada para poder procesarla o enviarla a la red.

LA INYECCIÓN DINÁMICA Y RESET

Aquí es donde se produce lo mas cool de la interactividad. Modificamos el **DOM** en tiempo real, en vivo y el navegador redibuja la interfase de forma instantánea ante nuestros ojos.

Línea 17

```
panelVista.innerHTML = '<span style="color: var(--color-alerta)">[CONECTANDO  
SONDAS ...]</span>';
```

Operación de escritura/Inyección dinámica.

Accedemos al contenedor de previsualización y usamos la propiedad **.innerHTML**. Esta instrucción borra cualquier contenido previo que tuviera ese nodo en la memoria RAM e inyecta una nueva etiqueta **** con estilos en tiempo real. Vemos el cambio en la pantalla en el mismo milisegundo.

Línea 18

```
panelTech.innerHTML = '<span style="color: var(--color-alerta)">[ENVIANDO  
PAQUETE AL SERVIDOR ... ]</span>';
```

Operación de escritura/Inyección dinámica.

Repetimos la operación sobre el panel técnico. Esto nos da un feedback visual inmediato y nos confirma que el programa reaccionó al clic, manteniendo la interfase viva mientras el paquete de datos se despacha hacia el backend.

Línea 19

```
inputObjetivo.value = "";
```

Operación de escritura de propiedad del **DOM**.

Volvemos a acceder al cuadro de texto de la interfase, pero esta vez no para leerlo, sino para asignarle un valor nuevo: un **string** completamente vacío ("). Esto limpia automáticamente el cuadro de la pantalla, dejándolo *reseteado* y listo para que podamos ingresar un nuevo dominio objetivo.

IMPACTO DE LOS RESULTADOS

Una vez que salimos a la red y el búnker central nos devuelve los datos procesados, ejecutamos la actualización definitiva de nuestro tablero.

Líneas 29

```
panelTech.innerHTML = `<span style="color: var(--color-terminal)">${datos.mensaje}</span>`;
```

Operación de Escritura/inyección dinámica con variables.

Reemplazamos el mensaje temporal de *enviando paquete* e inyectamos directamente en el DOM la propiedad **.mensaje** que nuestro servidor **backend** metió adentro del paquete **JSON (datos.mensaje)**. Envolvemos todo el bloque dentro de comillas invertidas (**backticks**) para fusionar el texto con la variable y lo teñimos de forma exacta con el color verde de nuestra terminal de comandos (**var(--color-terminal)**) - verde en el MVP... después veremos que se hace con las versiones nuevas-.

Línea 30

```
panelEnlaces.innerHTML = `<span style="color: var(--color-terminal)">Objetivo en servidor: ${datos.objetivo}</span>`;
```

Operación de Escritura/Inyección dinámica con variables.

Modificamos el tercer panel de la interfase de la misma manera, plasmando de forma visible en la pantalla la ruta analizada (datos.objetivo) que ya se encuentra almacenada de forma segura en la infraestructura del servidor.

Línea 32

```
panelTech.innerHTML = '<span style="color: red">[FALLO DE CONEXIÓN CON BÚNKER CENTRAL]</span>';
```

Operación de Escritura/Inyección para mitigación de errores.

Si la red se cae o nuestro servidor está apagado, el flujo salta al *bloque catch*. Esta línea toma el control del panel técnico e inyecta de forma instantánea una etiqueta en color rojo estricto (**color: red**). Así evitamos que la aplicación se congele y reportamos la alerta de que la infraestructura está fuera de línea.

Resumen de ROLES

Para consolidar el concepto de forma definitiva, podemos ver que en nuestro código manejamos el DOM con dos roles bien definidos:

Los Receptores (**getElementById, querySelector, .value**): *Escuchan nuestras acciones y leen lo que escribimos en el tablero.*

Los Actuadores (**.innerHTML, .value = ''**): *Toman los mandos de la memoria RAM del navegador, modifican los nodos y redibujan la interfase para mostrarnos las respuestas en vivo.*

****PARSING** (Para el que quiera saber...)

Parsing (o análisis sintáctico).

Para nosotros, el código es un conjunto de palabras, variables y funciones con sentido lógico. Pero para la computadora, inicialmente, nuestro archivo .html o .js es solamente una cadena de texto plana, un montón de caracteres ordenados uno detrás del otro, sin jerarquía ni significado.

El **parsing** es el proceso que toma ese texto plano, lo desarma, analiza su estructura y lo traduce a un formato que el procesador pueda entender y ejecutar.

LAS DOS ETAPAS DEL PROCESO

El parsing no ocurre de golpe. Se compone de dos fases consecutivas que transforman el texto crudo en una estructura lógica:

Fase A: ANÁLISIS LÉXICO O TOKENIZACIÓN

Antes de entender la gramática, el sistema necesita identificar las palabras individuales. El *analizador léxico* lee el texto carácter por carácter y lo agrupa en piezas básicas llamadas **tokens**.

Si nuestro código dice **const puerto = 3000;**, la máquina identifica cuatro tokens: una palabra clave (const), un nombre de variable (puerto), un operador de asignación (=) y un valor numérico (3000), descartando los espacios en blanco innecesarios.

Fase B: ANÁLISIS SINTÁCTICO (EL PARSING PROPIAMENTE DICHO)

Una vez que el sistema tiene los tokens sueltos, entra en juego el **parser**. Su trabajo es verificar si esos tokens respetan las reglas gramaticales del lenguaje (JavaScript, HTML, etc.).

Si escribimos `const puerto = 3000;`, el parser dice: *"Esto es correcto, la sintaxis cumple con la regla de declaración de variables"*.

Si escribimos `const = 3000 puerto`, el parser frena la ejecución, detecta que el orden rompe las reglas y nos lanza el famoso error de sintaxis en la terminal (*SyntaxError*).

EL RESULTADO DEL PARSING: UN ÁRBOL

Si los tokens son correctos y respetan la gramática, el parser construye una estructura jerárquica en la memoria RAM llamada **AST (Abstract Syntax Tree / Árbol de Sintaxis Abstracta)**.

Este árbol es fundamental porque establece las relaciones de causa, efecto y dependencia de nuestro código. Gracias al AST:

El motor de JavaScript sabe qué operaciones matemáticas se ejecutan primero.

Sabe qué variables están adentro de qué funciones.

Sabe exactamente qué bloques de código se tienen que ejecutar cuando se dispara un evento.

EL PARSER EN EL TARGET ANALYZER

En nuestra arquitectura del *Target Analyzer*, el parsing ocurre de forma viva en dos lugares críticos que ya programamos:

En el Frontend: Cuando el navegador recibe nuestro archivo **index.html**, realiza un parsing completo del texto y construye el **DOM** (el árbol de objetos en memoria RAM). Transforma las etiquetas de texto de los paneles en nodos interactivos vivos que podemos controlar con **JavaScript**.

En el Backend: Cuando configuramos la línea **app.use(express.json());** en nuestro `server.js`, estamos activando un parser de red. Este filtro agarra el texto plano estructurado en **JSON** que viaja por el cable desde el emisor, analiza su sintaxis y lo traduce automáticamente a un objeto de **JavaScript** real para que podamos leer las propiedades con **req.body.url** y asignarlas a nuestra constante **urlRecibida**.

PARSEAR es **romper el aislamiento del texto plano** para transformarlo en una estructura de datos jerárquica, validada y completamente operativa dentro de la memoria de la computadora.